MiniOSNova Technical Documentation

Course: IT3131
Institution: Central Philippine University
Project: MiniOSNova - Simulated Python Operating System
Group Name: SureballNa
Group Members: Mark Ellery Contreras, Tamara Nicole M. Dondiego, Shgian Dane Wilfred Delima, Kathy P. Flores, Justenn J. Diel

---

1. Introduction

MiniOSNova is a lightweight, command-line simulated operating system developed in Python. Created as a final project for IT3131, the program demonstrates how a real operating system manages core tasks behind the scenes without requiring actual hardware-level changes. Through a simple terminal interface, users can log in securely, manage virtual files and folders, start and stop background processes, and monitor or allocate system memory.

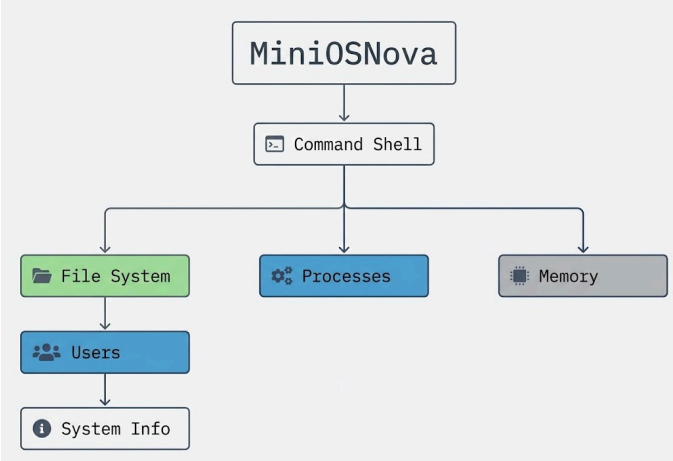
2. Project Objectives

The main goals of developing MiniOSNova were:

- To understand the core concepts of an operating system by building a working simulation in Python.
- To practice modular programming by splitting different system tasks (users, files, processes, and memory) into separate Python scripts.
- To implement a functional command-line shell (`MiniOSNova>`) that reads user inputs and routes them to the proper functions.
- To collaborate as a team using Git and GitHub for version control and documentation management.

3. System Architecture

MiniOSNova follows a modular architecture where the central command shell acts as the bridge between the user and the core system modules. When the user types a command, the Command Shell processes the text and calls the appropriate sub-system.
As shown in the architecture diagram, the system starts at the top with the main MiniOSNova environment, which runs the Command Shell. From the Command Shell, tasks branch out into File System management, Process management, Memory management, User authentication, and System Information display.



4. Program Structure

To keep our code organized and easy to debug as a group, we divided the program into five distinct Python files:

- **main.py:** The main entry point of the program. It handles clearing the screen, calling the login system, displaying the welcome banner, and running the `while True` loop that listens for user commands and routes them to the other files.
- **users.py:** Stores valid user accounts and passwords, runs the login authentication screen before allowing access to the OS, and handles the `whoami` command.
- **filesystem.py:** Manages all virtual files and folders in memory. It contains the functions for `dir`, `mkdir`, `create`, `read`, and `delete`.
- **processes.py:** Simulates background tasks and applications. It handles assigning unique Process IDs (PIDs) and contains the functions for `process`, `run`, and `kill`.
- **memory.py:** Simulates the computer's RAM. It tracks total, used, and free memory in Megabytes (MB) and runs the `memory`, `allocate`, and `free` commands.

5. File Management

Instead of creating real files on the computer's hard drive, MiniOSNova simulates a virtual file system inside Python using two data structures in `filesystem.py`:

- **Directories (`directories = [ ]`):** A Python list that stores the names of folders created by the user. When the user types `mkdir <name>`, the program checks if the folder already exists; if not, it appends the name to the list.
- **Files (`files = {}`):** A Python dictionary where the *key* is the filename and the *value* is the text content inside the file.
    - `create <name>` adds a new key to the dictionary and lets the user input text.
    - `read <name>` looks up the filename in the dictionary and prints its content.
    - `delete <name>` removes the filename key from the dictionary.
    - `dir` loops through both the `directories` list and the `files` dictionary to display everything currently stored in the system.

6. Process Management

Process management is handled inside `processes.py` to mimic how Windows Task Manager or Linux tracks running apps:

- When a user types `run <name>` (for example, `run calculator`), the system creates a simulated process with the status `RUNNING` and adds it to our active process list.
- Each process is given a unique **Process ID (PID)** using an incremental counter. Even if two instances of `calculator` are started, they receive different PIDs (e.g., PID 1 and PID 2).
- Just like a real OS, if a process is terminated using `kill <pid>`, its PID is retired for that session and the next new process continues from the next number in line (e.g., PID 5) rather than resetting back to 1.
- Typing `process` prints a formatted table showing the PID, Name, and Status of all currently active tasks.

7. Memory Management

The `memory.py` module simulates system RAM using integer variables that represent Megabytes (MB):

- The system starts with a fixed amount of Total Memory (e.g., 1024 MB) and 0 MB of Allocated Memory.
- When the user runs `allocate <MB>`, the program converts the input into an integer, checks if there is enough free memory available, and adds that amount to the allocated memory counter while subtracting it from free memory.
- When the user runs `free <MB>`, the program releases that amount of memory back into the free pool, making sure the user cannot free more memory than what is currently allocated.
- The `memory` command displays the current Total, Allocated, and Free memory values.

8. User Management

Security is simulated through `users.py` before the main command prompt ever appears:

- A dictionary stores pre-registered usernames and passwords.
- When `main.py` starts, it immediately calls `users.login_system()`. The user is prompted to enter a valid username and password.
- While typing the password, keystrokes are hidden so no one looking at the screen can see it.
- Once login succeeds, the username is saved in the `current_user` variable. This variable is later used when the user runs `whoami` or `systeminfo`.

9. Error Handling

Because users often make typos in a command-line interface, we built several layers of error handling so MiniOSNova does not crash:

- **Unknown Commands:** If a user types a command that does not exist (such as `kill5` without a space), the `else` block in `main.py` catches it and prints: `Unknown command. Type 'help' to see the available commands.`
- **Invalid Targets:** Trying to read a file that does not exist, kill a PID that is already terminated, or allocate more RAM than available triggers a clear warning message.

- **Keyboard Interrupts:** We wrapped the main loop in a `try-except` block that catches `KeyboardInterrupt` (when someone presses `Ctrl+C`) and reminds the user to type `exit` to shut down properly.

10. Testing

Our group tested MiniOSNova by running through every command in the VS Code terminal and trying to break the program:

- **Normal Flow Testing:** We logged in, checked `systeminfo` and `whoami`, created folders and text files, read them back, and deleted them.
- **Process & PID Testing:** We started multiple processes (`run calculator`, `run browser`, `run notepad`), ran duplicate app names to verify they got separate PIDs, killed active processes, and tried killing non-existent PIDs (like `kill 1` after it was already terminated) to confirm the error message worked.
- **Boundary & Input Testing:** We tested typing commands without arguments (just typing `mkdir` or `allocate` by itself), typing empty spaces at the prompt, and allocating/freeing memory repeatedly.

11. Problems Encountered

While developing and syncing the project as a student group, we ran into several common technical challenges:

Git Merge Conflicts and Non-Fast-Forward Errors: When our local repository and the online GitHub repository got out of sync, Git rejected our push and showed `HEAD` merge conflict markers across our Python files in VS Code. Since our local files had the complete working code, we solved this by using `git push -u origin main --force` and clearing the stuck merge state with `git merge --abort`.

- **Missing Input Crashes:** Early on, typing a command like `run` or `create` without a name after it caused Python to crash with an `IndexError: list index out of range`. We fixed this by checking `len(user_input) > 1` before reading `user_input[1]`.

12. Conclusion

Building MiniOSNova gave our group a much clearer understanding of how real operating systems work behind the scenes. We learned how a command shell parses user input, how files and directories can be represented using lists and dictionaries, how processes track unique incremental PIDs, and how memory allocation must be carefully checked to prevent errors. Beyond Python coding, we also learned practical lessons about teamwork, writing clear documentation, and troubleshooting real-world Git and GitHub issues in VS Code.